

Multiagent for Two Accounts — the ELI5 pages

Jaeuk Kim · Claude Code

2026-09-21

Two Houses for Claude

Why steps 3 to 8 look the way they do, in pictures.

Private house · the boss Team house · the helper Shared stuff

The big idea

Every login gets its own house.

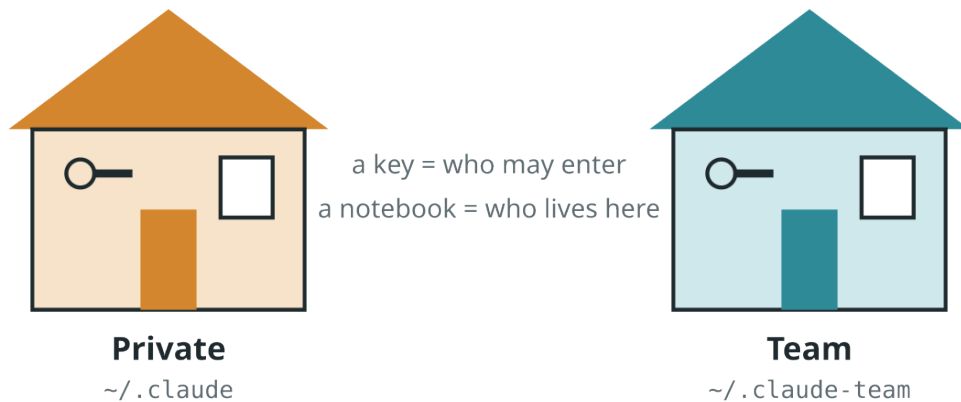


Figure 1: Everything about one login lives inside one folder. Two logins, two folders.

In grown-up words: Claude Code keeps token, settings, skills, agents, and memory under one directory, chosen by **CLAUDE_CONFIG_DIR**. Orca's account switcher is simply one such directory per account.

Step 3

Give the team house a short signpost.

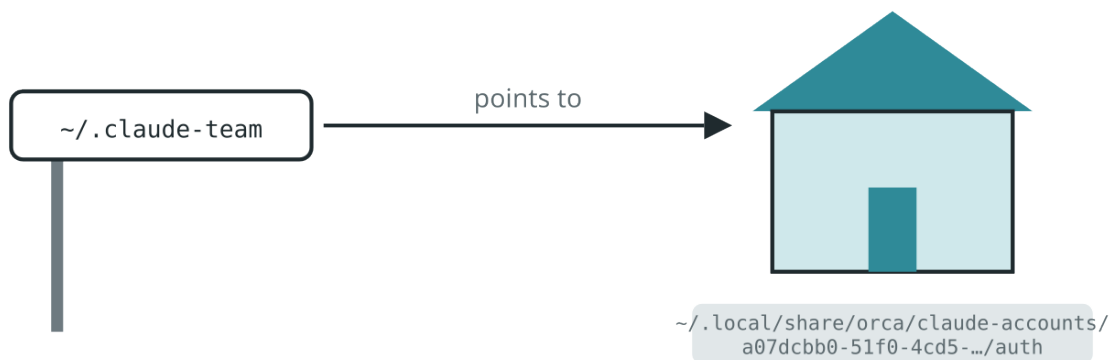


Figure 2: Orca built the house at a long, ugly address. The signpost is a name you can type.

In grown-up words: a symlink. Nothing is copied; the short path simply resolves to Orca's managed directory.

```
ln -s ~/.local/share/orca/claude-accounts/a07dcbb0-.../auth ~/.claude-team
```

Step 4

Share the furniture. Keep the keys.

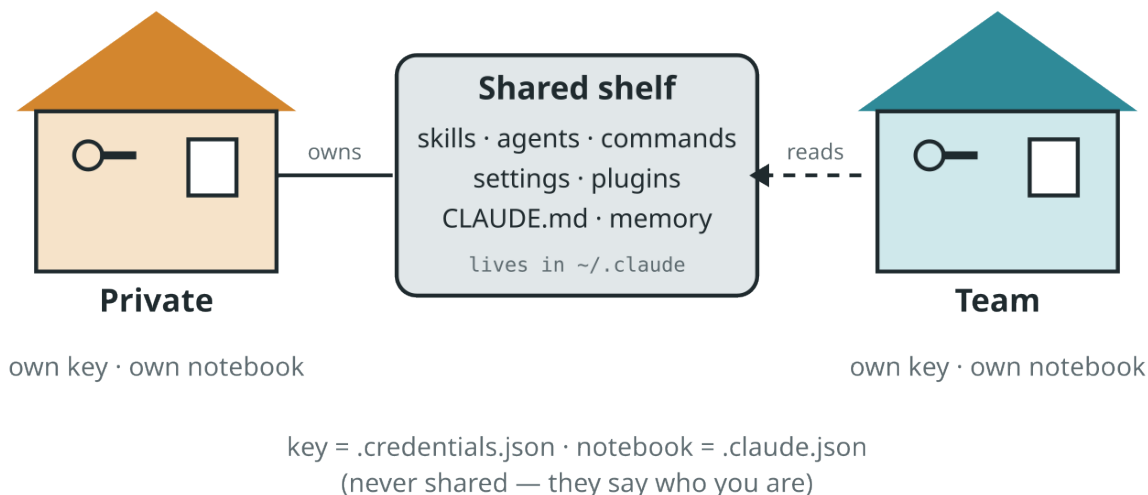


Figure 3: Change a skill once and both houses see it. But each house keeps its own key and its own notebook.

In grown-up words: inside `~/.claude-team`, **settings.json**, **CLAUDE.md**, **skills**, **agents**, **commands**, **plugins** and **projects** are symlinks into `~/.claude`. The token and the identity file stay per account.

Step 5

Phone numbers live in the notebook. Copy them.

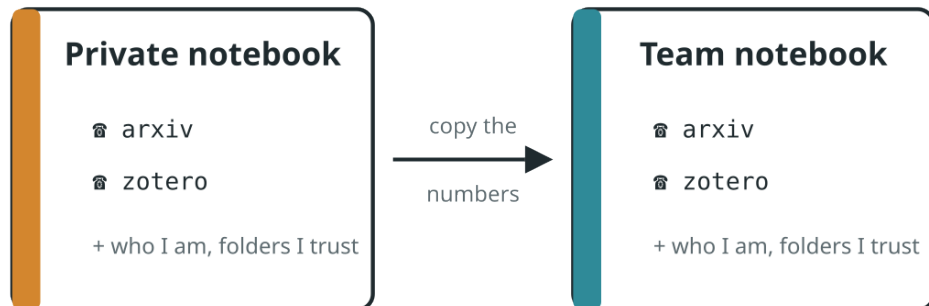


Figure 4: The notebook is private, so the numbers inside it don't come along with the shelf.

In grown-up words: user-scope MCP servers are stored in the per-account **.claude.json**, not in settings. Re-register them in the team dir.

```
CLAUDE_CONFIG_DIR=~/.claude-team claude mcp add-json -s user arxiv "$(jq -c .mcpServers.arxiv ~/.claude.json)"
```

Step 6

One door that always leads to the team house.

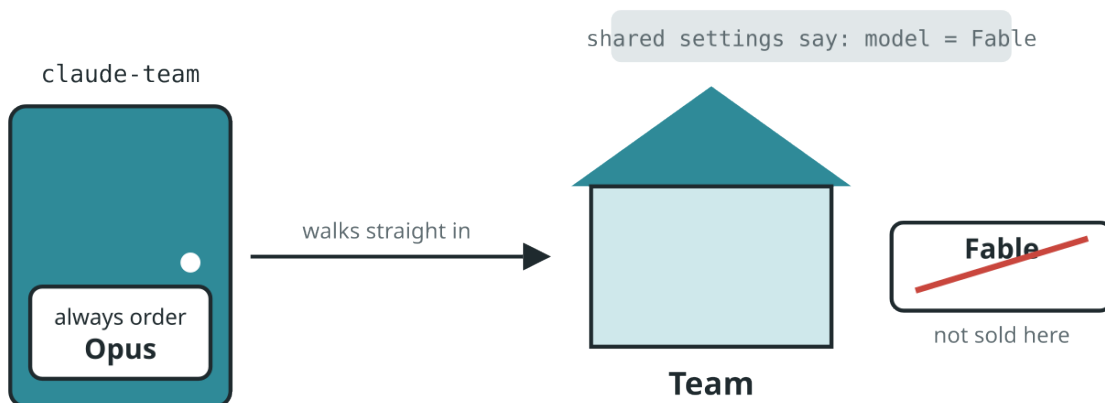


Figure 5: The shared shelf says "Fable" — but the team house can't buy Fable. So the door itself orders Opus every time.

In grown-up words: an alias sets the config dir and pins the model, because the shared **settings.json** defaults to Fable, which the team plan lacks.

```
alias claude-team='CLAUDE_CONFIG_DIR=$HOME/.claude-team claude --model opus'
```

!Never run `/model` inside the team house. It writes on the *shared* wall and would change the boss's default too.

Step 7

Peek inside once.

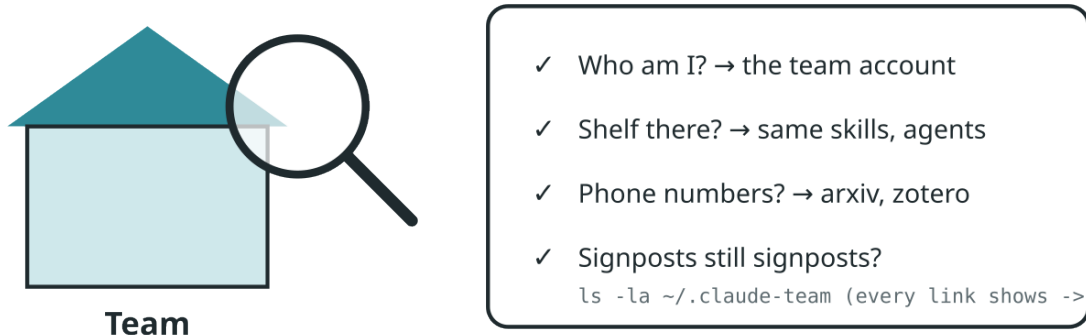


Figure 6: Walk in, check the four things, walk out. If a signpost turned into a copy, tell me.

In grown-up words: first interactive run — trust prompt, **/status**, **/skills**, **/mcp**, then confirm the symlinks survived. The one unverified point is whether a settings save replaces the symlink with a real file.

```
cd ~ && claude-team
```

Step 8

Helpers in your head use your wallet. Send a messenger instead.

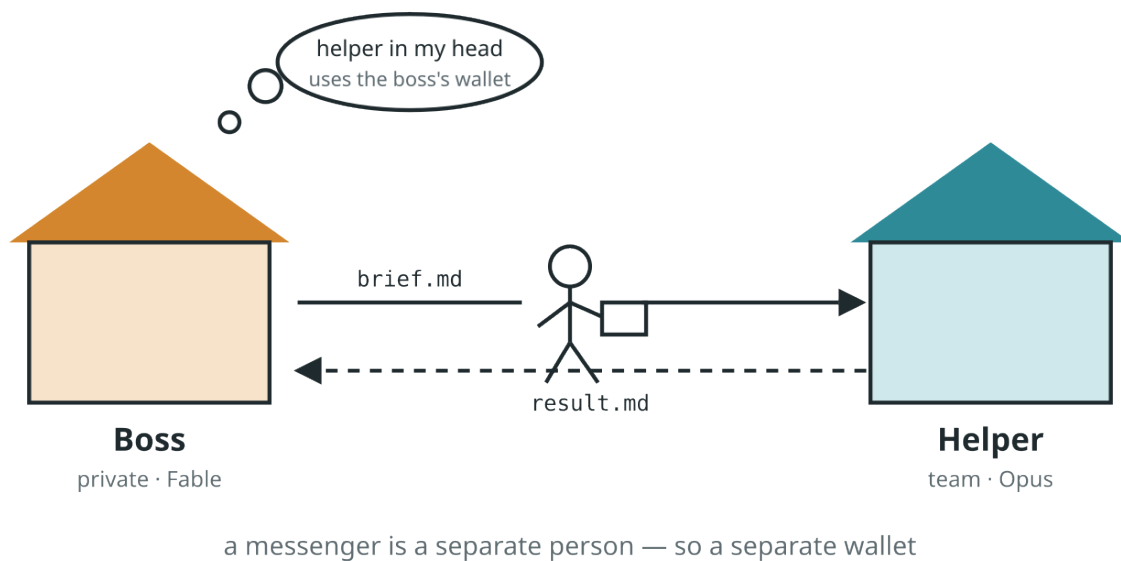


Figure 7: Anything the boss thinks up inside their own head is paid by the boss. Only a separate messenger can spend the team's money.

In grown-up words: in-process subagents (the Agent tool, **claude-main**, **deep-reasoner**) always bill the session's own account. The team account works only as a separate **claude-team -p** process.

```
claude-team -p "$(cat tasks/<task>/workers/claude-main/brief.md)" \  
> tasks/<task>/workers/claude-main/result.md
```

so: Boss = private house, Fable, thinks and decides. Helper = team house, Opus, does the long work. Same shelf of skills for both.

Team First, Boss Backup

The new rule for who pays for each job, in pictures.

Team wallet · spend first Boss wallet · backup, buys anything Rules & receipts

The big idea

Two wallets. Spend the team's first.

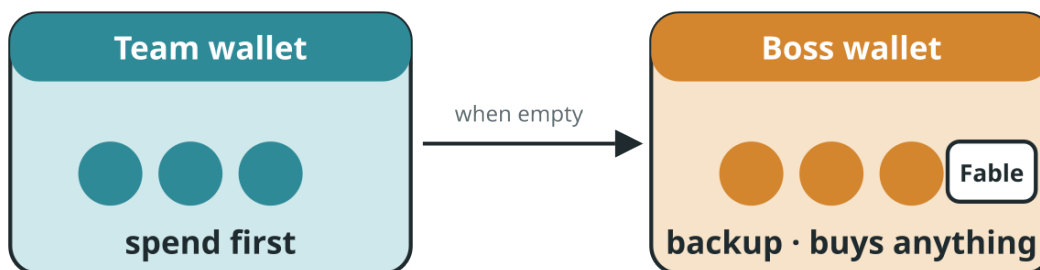


Figure 8: The team pays for every job it can. The boss pays only when the team wallet is empty — or when the job needs something only the boss can buy.

In grown-up words: team quota (KAIST, Opus/Sonnet/Haiku) is consumed first; the private account (Max, Fable) is the fallback and keeps every capability. Private is never restricted.

Step 1

Peek in the wallet before every job.

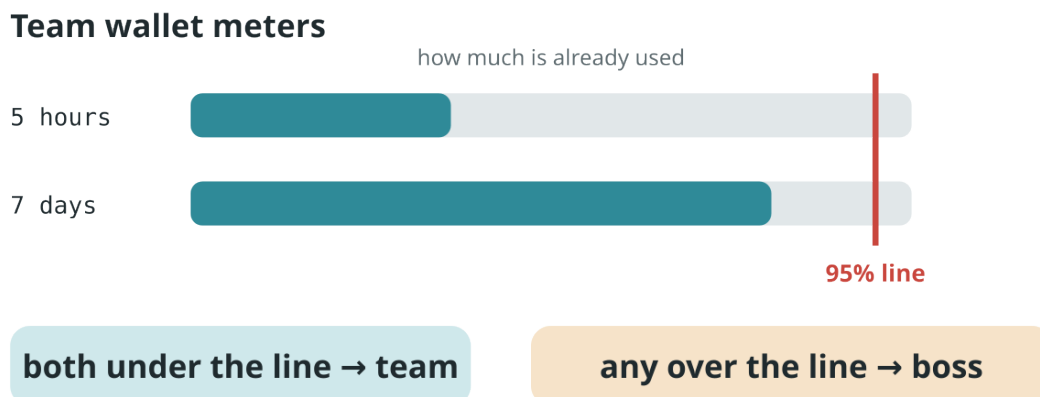


Figure 9: Look at the meters first. If either one is past the red line — or the meter can't be read — the boss pays.

In grown-up words: one usage-API probe with the team token; session, weekly-all, and the requested model's scoped window must all be below the cutoff (95%). Unknown, 401, or malformed → private, with a reason. Cached 60 s.

Step 2

The team only does looking-and-thinking jobs.



Figure 10: Team helpers may read, search, and review. Writing files and running commands stay with the boss — for now.

In grown-up words: CLI-dispatched Claude workers get **--tools Read,Grep,Glob**; write-capable and executable work stays on private until open items A1/A2 are solved. Private capability is unrestricted.

Step 3

A supervisor watches the messenger.

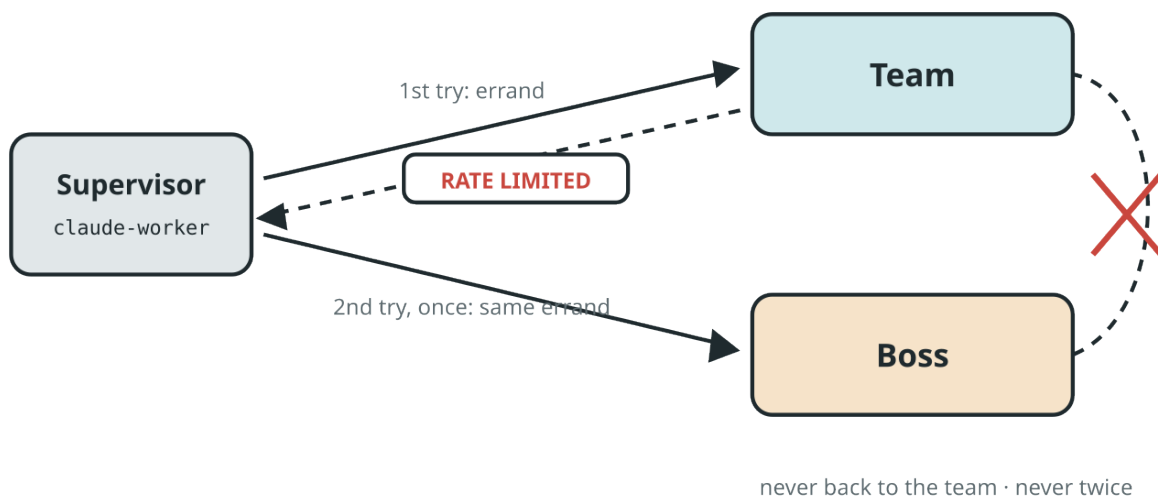
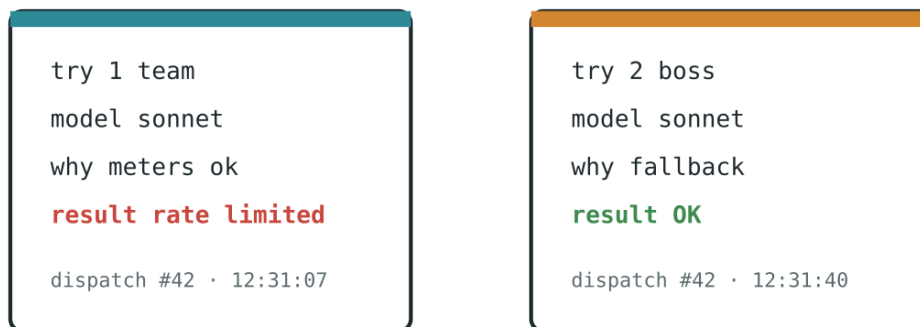


Figure 11: If the team helper comes back stamped "rate limited", the supervisor hands the same errand to the boss — once. Never back to the team, never a third time.

In grown-up words: a supervising subprocess (not **exec**) reads the structured **-p --output-format json** result; exactly one private retry, only on a positively classified rate-limit failure of a declared read-only run. Not for overload (529), interactive, resumed, or write-capable runs.

Step 4

Every try gets a receipt.



both slips are kept — you can always see who paid

Figure 12: Who ran it, which model, why, and how it ended — one slip per try, both kept.

In grown-up words: attempt-start/finish events (dispatch id, attempt #, backend, account label, model, reason, exit, error class) appended to the engine's **events.ndjson**; JSON lines on stderr for manual runs. Account label stays separate from author family.

Step 5

Two doors for you, one seat for the boss.

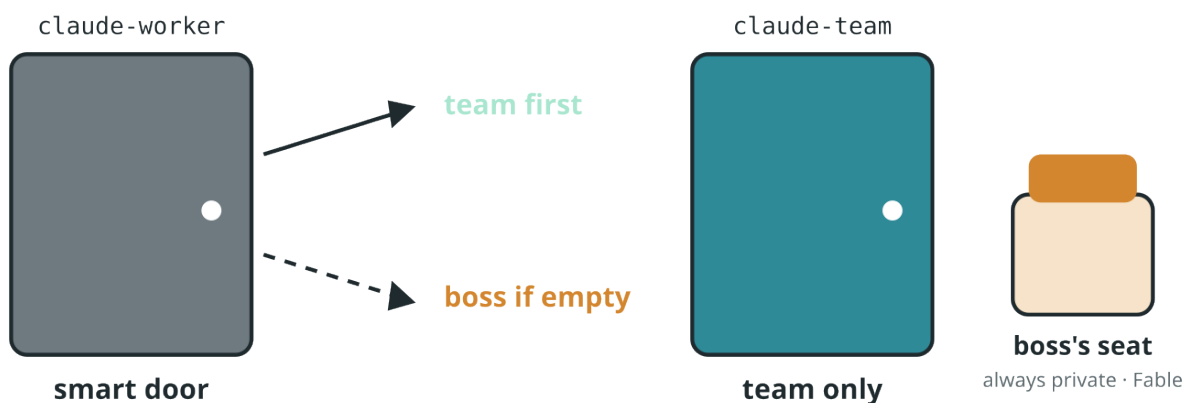


Figure 13: Use the smart door for helper jobs. Use the team-only door when you insist. The boss's own seat never moves.

```
claude-worker -p "$(cat brief.md)"    # team first, boss backup
claude-team   -p "$(cat brief.md)"    # forced team
```

In grown-up words: the conductor session stays on private; inside /multiagent, **claude-mid-team** is listed before **claude-mid** and the dispatcher takes one wallet snapshot per dispatch. Policy validation and dry-runs never go online.

Not now

What we deliberately left in the box.

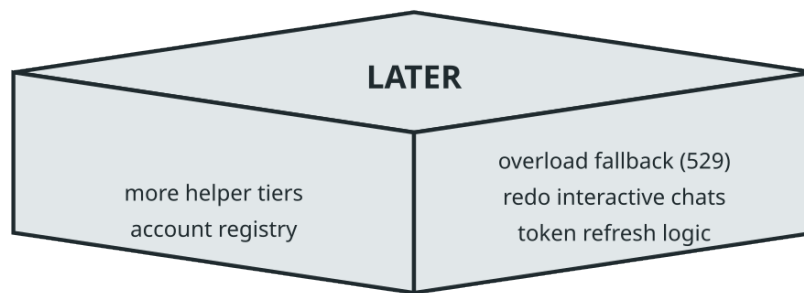
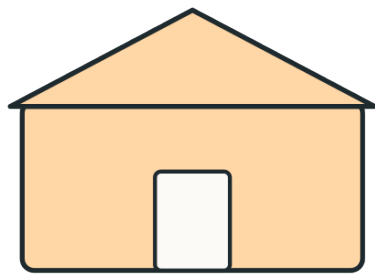


Figure 14: One helper pair, one probe, one retry rule. Everything else waits until this works.

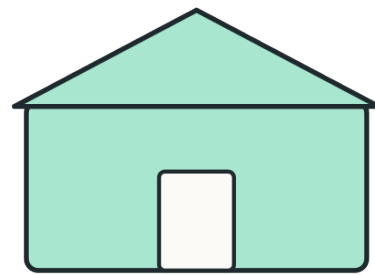
so: Team spends first. Boss backs up and can do anything. Peek, send, retry once, keep the receipts.

New House Rules

Two houses can do the work. Nine rules now decide which one, what it may touch, and who checks it.



Boss house
your own account · costly



Team house
the lab's account



helper



helper

Figure 15: Two houses can do the work. The rules decide which one gets each job.

1. Ask the Team house first

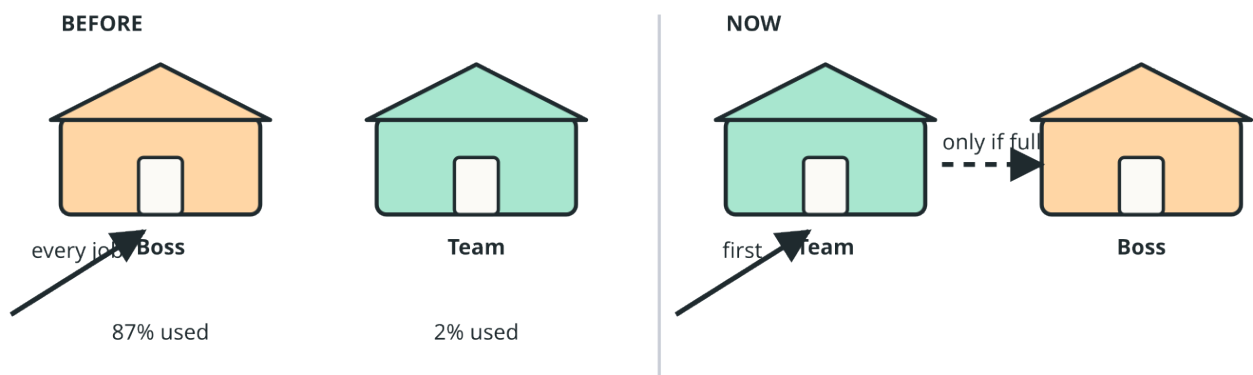


Figure 16: Writing jobs go to the Team house first, with the Boss house as backup when Team is full. Quick errands still go to the Codex house first, and checkers never come from Team.

2. Only mailed jobs can reach the Team house

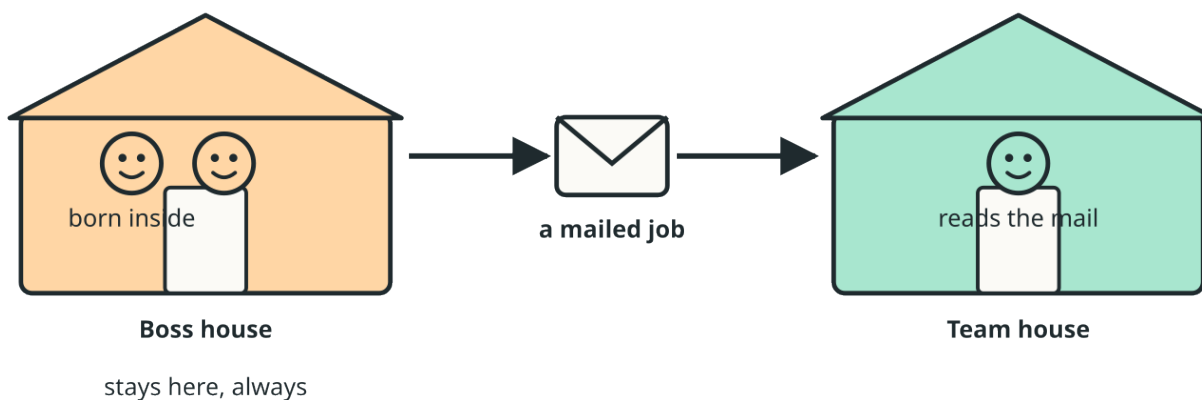


Figure 17: A helper born inside the Boss house always uses the Boss account. To use the Team house, the job must be mailed out (dispatch-worker).

3. A mailed helper can send a page back

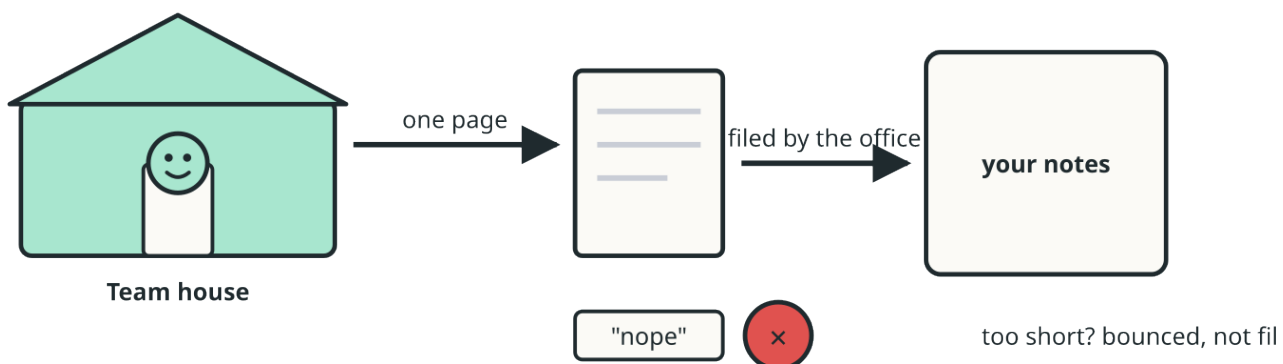


Figure 18: The helper writes a page and the office (the engine) files it. A one-word answer is bounced instead of filed over a real note.

4. A helper may write on ONE door

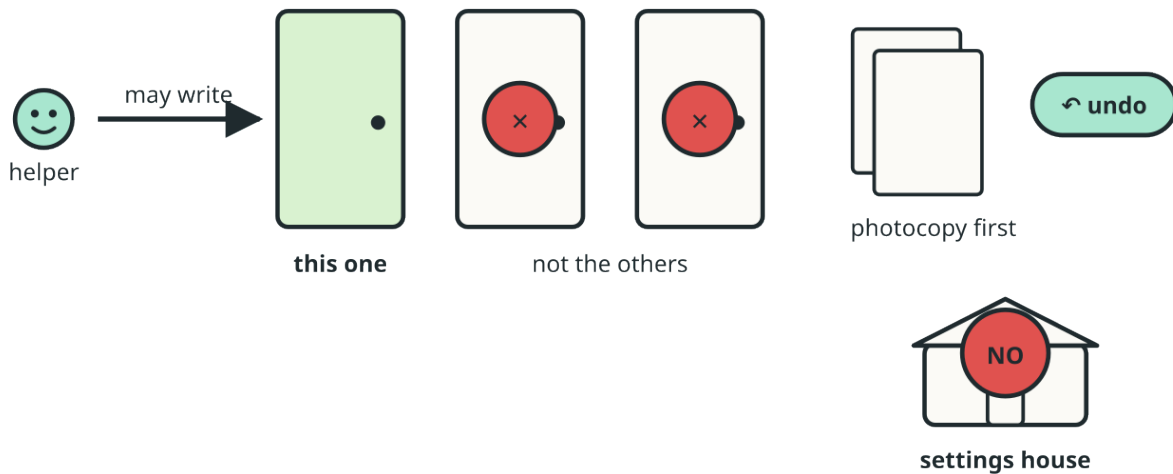


Figure 19: One file or one folder, named in advance. The office photocopies it first, so there is usually an undo — it refuses to undo if someone else changed the door since, or if the office could not check the door afterwards. The settings house (~/.claude) can never be a door.

5. Every writer signs the guestbook

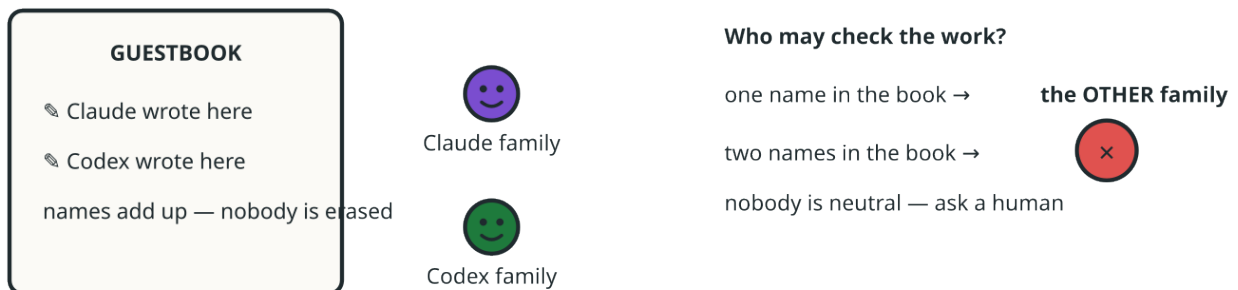


Figure 20: Everyone the office sees writing signs — mailed writers always, inside-born writers only when the door-keeper is on duty, and quick errands never. A checker must come from a family that did NOT write. If both families wrote, no automatic checker is allowed.

6. No signature, no checking

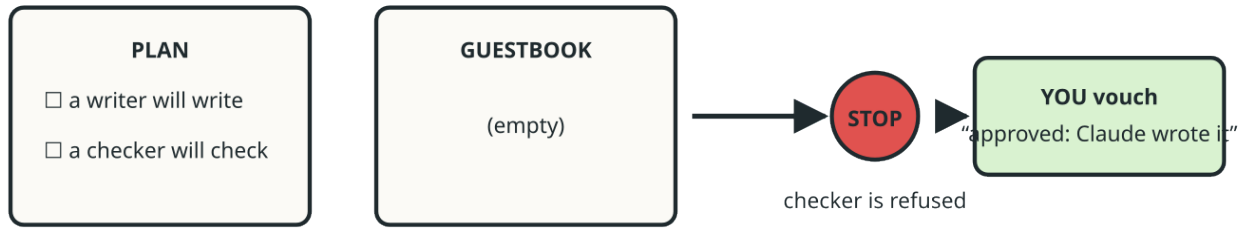


Figure 21: If the plan said someone would write but the guestbook is empty, checking is blocked. A helper saying “I wrote it” is not enough; your recorded OK, plus the helper’s statement, is what unblocks it.

7. One-door jobs by default — or write down why not

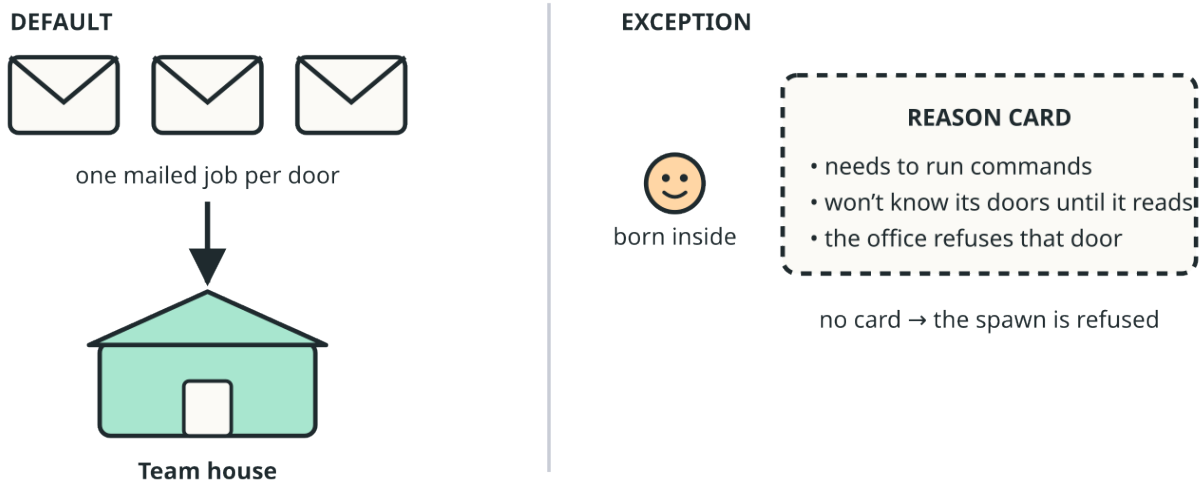


Figure 22: The rulebook says: cut big work into one-door jobs and mail them out. The office cannot cut the work for you — it only checks that a writer spawned inside the house has a reason card on the plan.

8. Helpers get a small rulebook, not your whole desk

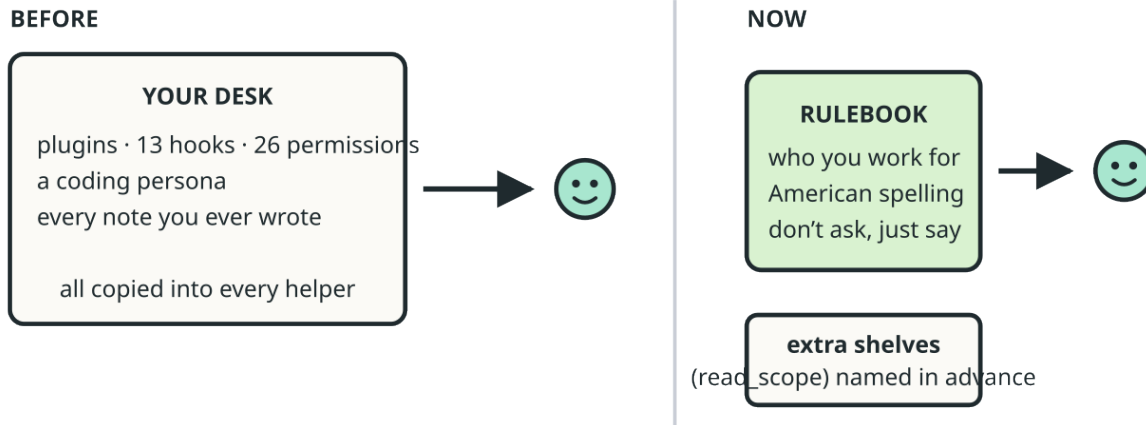
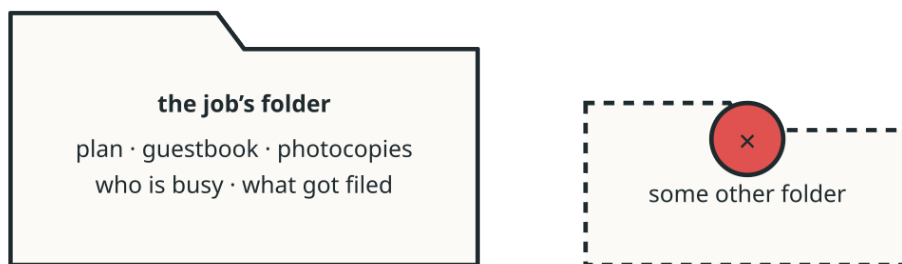


Figure 23: A Claude helper used to inherit everything on your desk. Now it gets a short rulebook, and only the reading shelves the plan names. About half the reading in one measurement, and none of the persona. (Codex helpers already read the whole library; shelves mean nothing to them.)

9. One folder per job



everyone reads and writes the same folder — so nothing hides

Figure 24: Everything about a job lives in its own folder, and a job may not point at a different one. That is how the checker, the office and the door-keeper all see the same guestbook.

What did not change

- A helper born inside the house still uses the Boss account, whatever it is told.
- Jobs that need doors scattered across the house, or a Claude helper that must run commands, still stay home. (A Codex helper can run commands, and one folder-door can hold many files.)
- A Codex helper still hands back a patch with no automatic way to land; its *pages* can be filed, its *code* waits for a human's small edits.
- The office checks that a reason card *exists*. It cannot tell whether it is true.